

# Namaste Python

## Visual Revision Guide — V3

Scenario-Based Interviews + System Design for Python Backend Developers



# NAMASTE PYTHON

## Concept-First Revision Guide

Understand • Remember • Apply • Interview Ready

### Python in One Line

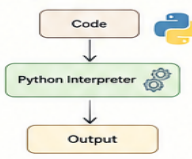
Python is a high-level, interpreted, general-purpose programming language with simple syntax and powerful libraries.

### ROADMAP

- Python Basics
- Data Types & Variables
- Operators
- Control Flow
- Functions
- \*args & \*\*kwargs
- Scope & LEGB Rule
- Closures & Decorators
- Iterators & Generators
- OOP (Object Oriented Programming)
- Exceptions
- Context Managers
- Modules & Packages
- Python Internals
- Async Python
- FastAPI Basics
- Pydantic
- SQLAlchemy Basics
- Testing with pytest
- Logging
- Interview Q & A
- Rapid Revision Sheet

### 1. PYTHON BASICS

- Python code is executed line by line.
- Indentation (spaces) is mandatory.
- Everything is an object in Python.



### 2. VARIABLES & DATA TYPES

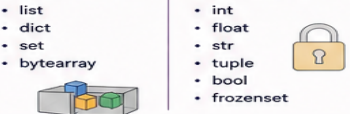
```
x = 10 # int
pi = 3.14 # float
name = "Alice" # str
is_valid = True # bool
items = [1, 2, 3] # list
person = {"name": "Bob"} # dict
unique = {1, 2, 3} # set
point = (1, 2) # tuple
```

| int | float | str  | bool |
|-----|-------|------|------|
| 10  | 3.14  | "Hi" | True |

| list    | tuple | set     | dict    |
|---------|-------|---------|---------|
| [1,2,3] | (1,2) | {1,2,3} | {'a':1} |

### 3. MUTABLE vs IMMUTABLE

| Mutable (Can be changed)                                                                        | Immutable (Cannot be changed)                                                                                              |
|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>list</li><li>dict</li><li>set</li><li>bytearray</li></ul> | <ul style="list-style-type: none"><li>int</li><li>float</li><li>str</li><li>tuple</li><li>bool</li><li>frozenset</li></ul> |



### 4. == vs is

**== (Value comparison)**  
a = [1, 2, 3]  
b = [1, 2, 3]  
print(a == b) # True

**is (Identity comparison)**  
a = [1, 2, 3]  
b = a  
print(a is b) # True

Use == to compare values  
Use is to check same object in memory

### 5. CONTROL FLOW

```
if / elif / else
x = 10
if x > 0:
    print("Positive")
elif x == 0:
    print("Zero")
else:
    print("Negative")

for loop
for i in range(3):
    print(i) # 0 1 2

while loop
i = 0
while i < 3:
    print(i)
    i += 1
```

### 6. FUNCTIONS

```
def greet(name):
    """This is a docstring"""
    return f"Hello, {name}!"

print(greet("Alice"))
```

**Types of Functions**

- Built-in (len(), print(), type())
- User-defined
- Lambda (anonymous)

**Lambda Example**

```
square = lambda x: x ** 2
print(square(4)) # 16
```

### 7. \*args & \*\*kwargs

**\*args (Non-keyword arguments)**

```
def add(*numbers):
    return sum(numbers)

print(add(1, 2, 3, 4)) # 10
```

**\*\*kwargs (Keyword arguments)**

```
def info(**data):
    for k, v in data.items():
        print(k, "=", v)

info(name="Alice", age=25)
```

### 8. SCOPE & LEGB RULE

Python looks for a variable in this order:

- L** Local (inside current function)
- E** Enclosing (inside enclosing function)
- G** Global (in global space)
- B** Built-in (python built-in names)

If not found at any level, NameError is raised.

### 9. CLOSURES & DECORATORS

**Closure Example**

```
def outer(msg):
    def inner():
        print(msg)
    return inner

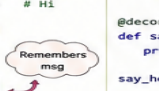
hello = outer("Hi")
hello() # Hi
```

**Decorator Example**

```
def decor(func):
    def wrapper():
        print("Before")
        func()
        print("After")
    return wrapper

@decor
def say_hello():
    print("Hello")

say_hello()
```



### 10. ITERATORS & GENERATORS

**Iterator**

```
it = iter([1,2,3])
print(next(it)) # 1
print(next(it)) # 2
print(next(it)) # 3
```

**Generator (using yield)**

```
def count(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for num in count(3):
    print(num)
```

Generators are memory efficient (lazy evaluation).

### 11. OOP IN PYTHON

**Class & Object**

```
class Person:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return f"Hi, I am {self.name}"

p = Person("Alice")
print(p.greet())
```

**Pillars of OOP**

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

### 12. EXCEPTIONS

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print("Cannot divide by zero")
else:
    print("No Exception")
finally:
    print("This will always run")
```

**Common Exceptions**

- ZeroDivisionError
- ValueError
- TypeError
- FileNotFoundError
- KeyError

### 13. CONTEXT MANAGERS

```
with open("file.txt", "r") as f:
    data = f.read()
# file is automatically closed
```

**Custom Context Manager**

```
class MyContext:
    def __enter__(self):
        print("Enter")
    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Exit")

with MyContext():
    print("Inside block")
```

### 14. PYTHON MEMORY MODEL (HIGH LEVEL)

Stack (Primitive)  
int, float, bool

Heap (Objects)  
list, dict, class, etc.

Objects in heap are referenced by variables in stack.

### 15. GIL (GLOBAL INTERPRETER LOCK)

- Allows only one thread to execute Python bytecode at a time.
- Good for I/O-bound tasks (not CPU-bound).
- Multiprocessing is used for CPU-bound tasks.

### 16. ASYNC IN PYTHON

```
import asyncio
async def main():
    print("Start")
    await asyncio.sleep(1)
    print("End")

asyncio.run(main())
```



★ **REMEMBER:** Practice code daily • Build projects • Solve problems • Revise regularly → You will master Python! 🚀

# Chapter 1 — Python Fundamentals: Scenario Interviews

These questions test whether you can apply Python concepts while debugging real code.

## Scenario 1 — Unexpected list changes

**Situation:** A function receives a list and appends an item. The caller says their original list changed unexpectedly.

**How I would approach it:** Check whether the function mutates the input. If mutation is not intended, copy at the boundary or return a new collection.

**Watch out for:** A shallow copy does not isolate nested mutable objects.

## Scenario 2 — Missing dictionary field

**Situation:** An API sometimes crashes with `KeyError` because old records do not contain an optional field.

**How I would approach it:** Use explicit validation or `.get()` for optional data and migrate old records when appropriate.

**Watch out for:** Do not hide genuine programming bugs with a broad exception handler.

## Scenario 3 — One million membership checks

**Situation:** You repeatedly check whether IDs exist in a large collection.

**How I would approach it:** Use a set when ordering is not required; average membership lookup is  $O(1)$ .

**Watch out for:** Sets use additional memory and elements must be hashable.

## Quick Interview Questions

### 1. Mutable vs immutable?

**Strong answer:** Mutable objects can change in place; immutable objects cannot be changed in place.

### 2. `==` vs `is`?

**Strong answer:** `==` compares values; `is` compares identity.

### 3. Why is `is None` preferred?

**Strong answer:** It explicitly checks identity and avoids custom equality behavior.

### 4. List vs tuple?

**Strong answer:** Both are ordered sequences, but lists are mutable and tuples are immutable.

# Chapter 2 — Functions, Scope, Closures & Decorators

Interviewers often give broken code and ask you to predict behavior or redesign it.

## Mutable default argument

**Simple:** Default arguments are evaluated once when the function is defined.

### Bug

```
def add_item(item, items=[]):
    items.append(item)
    return items

print(add_item('A'))
print(add_item('B'))
```

The same list can be reused across calls.

### Fix

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

A new list is created for each call unless one is explicitly supplied.

## Scenario — Add logging around service methods

**Situation:** Your team wants execution logging around many functions without repeating logging code.

**How I would approach it:** Create a decorator that logs entry, duration, and exceptions. Preserve metadata with ``functools.wraps``; use an async-aware wrapper for coroutine functions.

**Watch out for:** Do not block the event loop or log secrets.

## Scenario — Closure captures stale configuration

**Situation:** A closure captured a configuration value, but configuration later changes.

**How I would approach it:** Decide whether the desired behavior is snapshot-at-creation or live lookup. For live configuration, inject a config object/service instead of capturing a primitive.

**Watch out for:** Hidden state in closures can make debugging harder.

## Chapter Interview Questions

### 1. What is LEGB?

**Strong answer:** Local, Enclosing, Global, Built-in name resolution order.

### 2. What is a closure?

**Strong answer:** A function that retains access to variables from an enclosing scope.

### 3. Why use `functools.wraps`?

**Strong answer:** It preserves metadata such as the wrapped function's name and docstring.

### 4. What is late binding in closures?

**Strong answer:** The closure refers to the variable binding; a common fix is binding the current value as a default argument.

# Chapter 3 — Iterators, Generators & Data Processing

These scenarios are common when processing files, database rows, streams, or large API responses.

## Scenario — Process a 20 GB CSV

**Situation:** A server has 2 GB RAM but must process a 20 GB CSV.

**How I would approach it:** Stream rows or chunks, validate records incrementally, batch database writes, and use generators where useful.

**Watch out for:** Also consider malformed rows, batch transaction size, retries, checkpoints, and backpressure.

## Scenario — API returns 10 million records

**Situation:** An endpoint uses `.all()` and the process runs out of memory.

**How I would approach it:** Use pagination or streaming. For a large export, create an asynchronous export job and store the generated file.

**Watch out for:** Do not solve a design problem only by increasing RAM.

## Lazy generator pipeline

**Simple:** Generators allow stages to process one item at a time.

### Example

```
def read_numbers():
    for n in range(10):
        yield n

def squares(values):
    for n in values:
        yield n * n

result = squares(read_numbers())
print(next(result))
print(next(result))
```

Only requested values are produced.

## Chapter Interview Questions

### 1. Iterable vs iterator?

**Strong answer:** An iterable can produce an iterator; an iterator maintains state and produces the next item.

### 2. Why are generators memory efficient?

**Strong answer:** They produce values lazily rather than materializing the complete sequence.

### 3. What is StopIteration?

**Strong answer:** The signal used by an iterator to indicate that no more values are available.

### 4. What is backpressure?

**Strong answer:** A mechanism that prevents a producer from overwhelming a slower consumer.

# Chapter 4 — OOP & Design Scenarios

Focus on composition, interfaces, maintainability, and appropriate use of inheritance.

## Scenario — Multiple payment providers

**Situation:** The system supports one payment provider today and may add two more.

**How I would approach it:** Define a small common interface such as charge/refund/status, implement provider-specific adapters, and inject the provider into the service.

**Watch out for:** Avoid a huge base class full of provider-specific behavior.

## Scenario — God class

**Situation:** A 2,000-line service handles authentication, billing, reports, email, and database logic.

**How I would approach it:** Split responsibilities into cohesive modules/services. Keep orchestration separate from infrastructure and use abstractions where substitution is useful.

**Watch out for:** Do not create classes just for the sake of creating classes.

## Scenario — Multiple inheritance bug

**Situation:** Two parents define the same method and the child behaves unexpectedly.

**How I would approach it:** Inspect `ClassName.mro()` and understand cooperative `super()` calls.

**Watch out for:** Directly calling a specific parent can break a cooperative MRO chain.

## Chapter Interview Questions

### 1. Inheritance vs composition?

**Strong answer:** Inheritance models an is-a relationship; composition models has-a and is often more flexible.

### 2. What is SOLID?

**Strong answer:** Five principles aimed at maintainable, extensible object-oriented design.

### 3. What is dependency inversion?

**Strong answer:** High-level policy depends on abstractions rather than concrete infrastructure.

### 4. What is MRO?

**Strong answer:** The order Python follows to find methods/attributes, especially important in multiple inheritance.

# Chapter 5 — Exceptions, Reliability & Debugging

Production interviews focus on safe recovery rather than simply catching exceptions.

## Scenario — Intermittent database timeout

**Situation:** A database operation occasionally times out.

**How I would approach it:** Retry only transient errors, use bounded retries with exponential backoff and jitter, and make the operation idempotent before retrying.

**Watch out for:** Retries can amplify an outage if they are unlimited or immediate.

## Scenario — API returns HTTP 500

**Situation:** Users see 500 errors while logs contain only 'Something went wrong'.

**How I would approach it:** Log the exception and stack trace internally, return a safe response, include a correlation ID, and classify failures into validation/business/dependency/unexpected errors.

**Watch out for:** Never expose stack traces, SQL, credentials, or internal infrastructure details.

## Scenario — Partial batch failure

**Situation:** Record 631 of 1,000 fails during a batch.

**How I would approach it:** Choose atomic all-or-nothing or explicit partial-success processing. Track per-record status and make processing idempotent.

**Watch out for:** A single global try/except can hide which records failed.

## Chapter Interview Questions

### 1. What is idempotency?

**Strong answer:** Repeating the same operation results in the same intended final state.

### 2. How should retries be designed?

**Strong answer:** Retry transient failures with limits, backoff, jitter, and idempotency.

### 3. Why avoid bare except?

**Strong answer:** It can hide unexpected programming errors.

### 4. Where should exception handling live?

**Strong answer:** At meaningful recoverable-operation boundaries and at the API/application boundary for final error mapping.

# Chapter 6 — Async Python & Concurrency Scenarios

Know when to use asyncio, threads, processes, or a durable worker queue.

## Scenario — FastAPI endpoint is slow under load

**Situation:** An async endpoint calls a synchronous SDK that takes two seconds.

**How I would approach it:** Replace it with an async client where possible, or deliberately move blocking work to a thread/worker pool.

**Watch out for:** async def does not make blocking code non-blocking.

## Scenario — Call five independent APIs

**Situation:** An endpoint needs five unrelated external API results.

**How I would approach it:** Run independent calls concurrently using tasks/gather, with per-call timeouts and controlled concurrency.

**Watch out for:** Do not create unlimited outbound concurrency.

## Scenario — CPU-heavy PDF/OCR processing

**Situation:** A request performs 30 seconds of CPU-heavy processing.

**How I would approach it:** Move it to a durable background worker/job queue and return a job ID.

**Watch out for:** A simple in-process background task is not a replacement for a durable queue for heavy work.

## Concurrency decision guide

**Simple:** Pick the model based on workload.

### I/O-bound

```
asyncio / async clients  
or threads for blocking I/O
```

Good when most time is spent waiting.

### CPU-bound

```
multiprocessing / worker processes
```

Useful when computation dominates.

### Long durable jobs

```
queue + workers
```

Supports retries, persistence, monitoring, and independent scaling.

## Chapter Interview Questions

### 1. Threading vs asyncio?

**Strong answer:** Both can support I/O concurrency; asyncio uses cooperative scheduling and requires async-compatible operations.

### 2. Why can async code still be slow?

**Strong answer:** Blocking calls or CPU-heavy work can block the event loop.

### 3. When use multiprocessing?

**Strong answer:** For CPU-heavy work where process-level parallelism is useful.

### 4. How prevent unlimited concurrency?

**Strong answer:** Use semaphores, bounded pools, queue limits, or rate limiting.



# Chapter 7 — FastAPI Production Scenarios

Think about request lifecycle, dependency injection, authentication, authorization, validation, and long-running work.

## Scenario — Client supplies company\_user\_id

**Situation:** An API accepts an ownership ID from the request body, allowing a caller to try another user's ID.

**How I would approach it:** Get the authenticated user from a dependency such as `Depends(get_current_company_user)`` and authorize access to the requested resource.

**Watch out for:** Authentication identifies the caller; authorization determines what they may access.

## Scenario — Slow endpoint

**Situation:** An endpoint performs authentication, multiple DB queries, an external call, and large serialization.

**How I would approach it:** Measure each stage. Optimize query count/indexes, loading strategy, external I/O, caching, and payload size.

**Watch out for:** Do not optimize without measurements.

## Scenario — Large PDF upload and processing

**Situation:** Users upload large documents for OCR/chunking/embedding.

**How I would approach it:** Store the original in object storage, save metadata, enqueue a durable task, process asynchronously, and expose task status.

**Watch out for:** Do not hold large files or long-running work inside the HTTP request.

## Scenario — API versioning

**Situation:** Old mobile clients need the old response while a new frontend requires a breaking change.

**How I would approach it:** Version the API or preserve backward compatibility. Track old-version usage before deprecation.

**Watch out for:** Breaking response changes can be especially difficult to roll out safely.

## FastAPI Interview Questions

### 1. Depends vs middleware?

**Strong answer:** Dependencies are explicit, composable request-scoped components; middleware wraps the broader request/response lifecycle.

### 2. Authentication vs authorization?

**Strong answer:** Authentication answers who you are; authorization answers what you are allowed to do.

### 3. How handle long-running work?

**Strong answer:** Use a durable job/queue and return a task identifier.

### 4. How improve FastAPI performance?

**Strong answer:** Measure first; then optimize DB, blocking I/O, network calls, serialization, caching, and concurrency.



# Chapter 8 — SQLAlchemy & Database Scenarios

Focus on transaction boundaries, indexes, query performance, concurrency, and connection management.

## Scenario — N+1 queries

**Situation:** One query loads 100 users and then one query is executed for each user's orders.

**How I would approach it:** Inspect SQL and use `selectinload/joinedload` as appropriate. Return only required fields where possible.

**Watch out for:** Do not eager-load every relationship blindly.

## Scenario — Duplicate order creation

**Situation:** Two requests simultaneously create the same unique order.

**How I would approach it:** Enforce uniqueness at the database level and handle the integrity error safely. Application-only checks are race-prone.

**Watch out for:** Critical invariants belong in database constraints.

## Scenario — Multi-write transaction

**Situation:** Creating an order requires several related inserts and one insert fails.

**How I would approach it:** Use a transaction when atomicity is required. If an external payment system is involved, use an outbox/saga/idempotency approach because the external system is not part of the DB transaction.

**Watch out for:** Database transactions cannot magically roll back external APIs.

## Scenario — Connection pool exhaustion

**Situation:** Production runs out of database connections.

**How I would approach it:** Inspect pool settings, session lifecycle, leaked sessions, long transactions, and concurrency. Close sessions correctly.

**Watch out for:** Increasing pool size without checking DB capacity can make the problem worse.

## SQLAlchemy Interview Questions

### 1. What causes N+1?

**Strong answer:** Lazy-loading related data repeatedly after fetching parent rows.

### 2. How prevent duplicate data?

**Strong answer:** Use unique constraints/indexes and handle conflicts.

### 3. What is a transaction?

**Strong answer:** A unit of DB work providing atomicity and consistency guarantees.

### 4. Why avoid long transactions?

**Strong answer:** They hold locks/resources longer and increase contention.

### 5. flush vs commit?

**Strong answer:** flush sends pending SQL within the transaction; commit finalizes it.

# Chapter 9 — System Design Fundamentals

A strong answer is structured: requirements → estimates → API/data → architecture → scaling → failure handling → trade-offs.

|    |                                                                             |
|----|-----------------------------------------------------------------------------|
| 1  | Functional requirements — what must the system do?                          |
| 2  | Non-functional requirements — latency, availability, consistency, security. |
| 3  | Scale estimates — RPS, peak traffic, storage, bandwidth.                    |
| 4  | API design — endpoints, payloads, pagination, idempotency.                  |
| 5  | Data model — tables/collections, indexes, relationships.                    |
| 6  | Architecture — API, cache, DB, queue, workers, storage.                     |
| 7  | Scaling — stateless replicas, load balancer, read replicas, partitioning.   |
| 8  | Reliability — timeouts, retries, DLQ, backups, failover.                    |
| 9  | Observability — logs, metrics, traces, alerts.                              |
| 10 | Trade-offs — explain why your design is appropriate.                        |

## System Design 1 — URL Shortener

**Situation:** Design a Bitly-like service that creates short URLs and redirects users.

**How I would approach it:** Use stateless API servers behind a load balancer, a mapping DB, Redis for hot redirects, unique ID/base62 generation, rate limiting, and asynchronous analytics.

**Watch out for:** Protect against collisions, abuse, malicious destinations, and cache stampedes.

## System Design 2 — File Upload & Processing

**Situation:** Users upload large PDFs and the system must extract text, chunk, embed, index, and expose processing status.

**How I would approach it:** Client/API → object storage → metadata DB → durable queue → workers → vector DB. Use signed uploads, task states, retries, idempotency, and monitoring.

**Watch out for:** Separate upload from processing; do not keep the request open during OCR/embedding.

## System Design 3 — Notification Service

**Situation:** Users can receive email, SMS, push, or WhatsApp notifications.

**How I would approach it:** Persist intent/outbox, publish channel jobs, use channel workers, retries/backoff, provider failover, deduplication, rate limits, templates, and preferences.

**Watch out for:** Exactly-once delivery is difficult; design for at-least-once processing with idempotent consumers.

## System Design 4 — News/Company Feed

**Situation:** Company news arrives continuously and must be searchable quickly.

**How I would approach it:** Ingest → normalize → deduplicate → store canonical records → index → cache popular queries. Partition by company/date as scale grows.

**Watch out for:** Freshness, source reliability, deduplication, and indexing are key concerns.

# Chapter 10 — Advanced System Design Interview Questions

For each question, explain the architecture and then discuss bottlenecks and failure cases.

## Question Set

### 1. Design a distributed rate limiter.

**Strong answer:** Discuss token bucket/sliding window, Redis/distributed state, atomic operations, TTLs, per-user/IP/API-key limits, and failure behavior.

### 2. Design a distributed job queue.

**Strong answer:** Discuss producers, durable broker, workers, acknowledgements, visibility timeout, retries, backoff, DLQ, idempotency, and scaling.

### 3. Design a caching layer.

**Strong answer:** Discuss cache-aside, TTL, invalidation, hot keys, cache stampede protection, Redis clustering, and cache failure.

### 4. Design an authentication system.

**Strong answer:** Discuss password hashing, sessions/JWT, refresh tokens, revocation, key rotation, MFA, rate limits, and authorization.

### 5. Design an analytics pipeline.

**Strong answer:** Discuss ingestion, queue/stream, consumers, batch/stream processing, warehouse, partitioning, deduplication, and late events.

### 6. Design a RAG backend.

**Strong answer:** Discuss document ingestion, storage, parsing, chunking, embeddings, vector search, metadata filters, reranking, LLM generation, caching, and evaluation.

### 7. Scale FastAPI from 100 to 10,000 RPS.

**Strong answer:** Measure first; then scale stateless replicas, optimize DB/indexes, cache hot reads, move heavy work to queues, tune connection pools, and introduce partitioning/read replicas when justified.

### 8. How do you make an API idempotent?

**Strong answer:** Use an idempotency key tied to a unique operation record/result and return the original result for retries.

### 9. Horizontal vs vertical scaling?

**Strong answer:** Vertical adds resources to one machine; horizontal adds instances and usually requires stateless/distributed design.

### 10. How do you design for failure?

**Strong answer:** Use timeouts, bounded retries, circuit breakers, queues, replication, backups, health checks, observability, and graceful degradation.

# Chapter 11 — System Design Deep-Dive: RAG Document Processing

A useful scenario for Python/FastAPI + RAG backend interviews.

| Component      | Responsibility                                              |
|----------------|-------------------------------------------------------------|
| Frontend       | Upload document, display status, send chat query.           |
| FastAPI        | Auth, validation, document metadata, chat APIs.             |
| Object Storage | Original PDFs/documents.                                    |
| SQL DB         | Users, KBs, documents, tasks, permissions, status.          |
| Queue          | Durable handoff from API to workers.                        |
| Worker         | Download → parse → OCR → chunk → embed → index.             |
| Vector DB      | Semantic search + metadata filtering.                       |
| Reranker       | Improve relevance of retrieved chunks.                      |
| LLM            | Generate answer from selected context.                      |
| Redis          | Caching, rate limits, short-lived status where appropriate. |
| Observability  | Logs, metrics, traces, processing/error tracking.           |

## Failure — Worker crashes at 70%

**Situation:** A worker crashes after writing some chunks but before marking the task completed.

**How I would approach it:** Use task states, idempotent writes, deterministic chunk IDs/upserts, retryable jobs, and checkpoints for very large documents.

**Watch out for:** Avoid duplicate vectors and inconsistent SQL/vector-store state.

## Scale — 1,000 documents arrive simultaneously

**Situation:** The API must stay responsive during a processing spike.

**How I would approach it:** Persist metadata and enqueue quickly. Scale workers based on queue depth, control provider concurrency, and batch embeddings where appropriate.

**Watch out for:** API servers and workers have different scaling characteristics.

## Quality — Retrieval is poor

**Situation:** Users complain that the generated answers are bad even though the LLM is capable.

**How I would approach it:** Evaluate retrieval separately. Tune chunking, metadata filters, query rewriting, candidate count, reranking, hybrid retrieval, and context assembly before changing the LLM.

**Watch out for:** Separate retrieval problems from generation problems.

## RAG System Design Questions

### 1. Why use a queue?

**Strong answer:** It decouples HTTP requests from long-running processing, absorbs spikes, supports retries, and enables independent worker scaling.

### 2. How avoid duplicate processing?

**Strong answer:** Use document/task idempotency, unique constraints, deterministic IDs, and upsert semantics.

### 3. How handle scanned PDFs?

**Strong answer:** Detect missing text, run OCR, validate extracted text, then continue through chunking and embedding.

### 4. How improve retrieval?

**Strong answer:** Measure retrieval independently and tune chunking, filters, query rewriting, top-k/fetch-k, reranking, and hybrid retrieval.

**5. What should be sync vs async?**

**Strong answer:** Metadata/auth/validation can be synchronous; OCR, embedding, indexing, and large exports should usually be asynchronous.

# Chapter 12 — Interviewer Follow-Up Questions

Interviewers often challenge your first answer. Be ready for these.

## Follow-Up Set

### 1. What if Redis goes down?

**Strong answer:** Explain fallback behavior, whether the feature fails open/closed, and how degraded performance is handled.

### 2. What if the DB becomes slow?

**Strong answer:** Use timeouts, inspect indexes/queries, reduce load, and degrade non-critical features.

### 3. What if the queue is full?

**Strong answer:** Apply backpressure, reject/defer work, autoscale consumers, and alert on queue depth.

### 4. What if a job executes twice?

**Strong answer:** Make consumers idempotent and deduplicate side effects.

### 5. What if an external API is unavailable?

**Strong answer:** Timeout, bounded retry, circuit breaker/fallback, and preserve work for later retry.

### 6. How do you monitor the system?

**Strong answer:** Latency, throughput, error rate, queue depth, DB pool use, cache hit rate, dependency latency, and business metrics.

### 7. Where is the bottleneck?

**Strong answer:** State an estimate, identify likely constraints, then explain how you would measure and validate them.

### 8. What is the consistency model?

**Strong answer:** State whether strong, transactional, or eventual consistency is required for each path.

### 9. Where is the single point of failure?

**Strong answer:** Identify it and explain replication, failover, or why the risk is acceptable.

### 10. What would you build first?

**Strong answer:** Start with the simplest architecture that meets requirements; add complexity when measurements justify it.

# Final — 30-Minute Interview Revision Checklist

For every scenario, answer: clarify requirements → identify constraints → propose solution → failure cases → scaling → trade-offs.

|                      |                                                                                                                           |
|----------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Python</b>        | Mutable/immutable; ==/is; LEGB; args/kwargs; decorators; generators; OOP; MRO; exceptions; context managers; memory; GIL. |
| <b>Async</b>         | Coroutine; await; event loop; task; gather; blocking code; concurrency limits.                                            |
| <b>FastAPI</b>       | Depends; Pydantic; auth vs authorization; async endpoints; queues; API versioning.                                        |
| <b>Database</b>      | Transactions; flush/commit; indexes; N+1; loading strategies; pools; uniqueness/concurrency.                              |
| <b>Distributed</b>   | Queues; retries; idempotency; rate limits; caching; timeouts; circuit breakers; DLQ.                                      |
| <b>System Design</b> | Requirements → estimates → APIs → data → architecture → scaling → failures → observability → trade-offs.                  |
| <b>RAG</b>           | Ingestion → storage → parsing → chunking → embeddings → vector search → reranking → LLM → evaluation.                     |

**Golden rule:** In system design, do not start with technology names. Start with requirements and constraints, then choose the technology that solves the actual bottleneck.